MOBILE APPLICATION DEVELOPMENT
BSc (Hons) in Information Technology

Android Database Connectivity

Hands-On Lab Guide: SQLite, Room & Cloud (Firebase)

This lab guide accompanies the “Android Database Connectivity” lecture. It turns the lecture's three storage layers — SQLite, Room, and Firebase — into three progressively harder, hands-on Android Studio activities, each with numbered build steps, working Kotlin code, and practice exercises you complete on your own.

> **How to use this guide**
> Work through the activities in order — Activity 2 reuses the table design from Activity 1, and Activity 3 reuses the User/Student model from Activities 1–2. Each activity ends with a short “Practice on your own” section and a checklist; use these to self-assess before moving on.

Prerequisites

- Android Studio (Hedgehog or later) with an emulator or physical device running API 24+.
- Basic Kotlin syntax: classes, data classes, functions, and coroutines/lifecycleScope.
- A free Google/Firebase account for Activity 3 (Cloud Firestore).
- A new Android project per activity is recommended (Empty Views Activity template, Kotlin, View Binding enabled).

What you will build

- Activity 1 (SQLite): a Registration & Login screen backed by a hand-written SQLiteOpenHelper.
- Activity 2 (Room): the same data, rebuilt with Room's Entity → DAO → Database pattern, displayed in a RecyclerView.
- Activity 3 (Cloud): a Student Directory synced through Cloud Firestore, matching the lecture's mini case study.
- Capstone practice: wiring Room + Firestore together behind a Repository for offline-first sync.

Activity 1 · SQLite: Registration & Login

Storage layer	Raw SQLite via SQLiteOpenHelper
Builds on	Slides 6–11 — SQLite architecture & CRUD
You will practice	Creating tables, INSERT with ContentValues, parameterized SELECT, Cursor handling
Estimated time	45–60 minutes

Objective

Build a two-screen app (Register, Login) that stores users in a local SQLite table using a hand-written SQLiteOpenHelper — no libraries, no abstraction — so you feel the boilerplate that Room later removes.

Step-by-step build

1. Create a new project "SQLiteAuthDemo" (Empty Views Activity, Kotlin, min SDK 24).
2. Add two layouts: activity_register.xml (username, email, password EditTexts + a Register button) and activity_login.xml (email, password EditTexts + a Login button). Reuse the same field IDs pattern in both so the Kotlin code below maps directly.
3. Create DBHelper.kt — a subclass of SQLiteOpenHelper that creates the users table on first run and handles version upgrades:

```
class DBHelper(context: Context) :
    SQLiteOpenHelper(context, "UserDB", null, 1) {

    override fun onCreate(db: SQLiteDatabase) {
        val sql = """
            CREATE TABLE users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT, email TEXT, password TEXT
            )
        """
        db.execSQL(sql)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS users")
        onCreate(db)
    }
}
```

4. In RegisterActivity, open a writable database from DBHelper and insert the form values using ContentValues (never build the SQL string by hand):

```
val dbHelper = DBHelper(this)
val db = dbHelper.writableDatabase

val values = ContentValues().apply {
    put("username", usernameEditText.text.toString())
    put("email", emailEditText.text.toString())
```

```
    put("password", passwordEditText.text.toString())
}
val newRowId = db.insert("users", null, values)

if (newRowId != -1L) {
    Toast.makeText(this, "Registered!", Toast.LENGTH_SHORT).show()
    finish()
}
```

1. In LoginActivity, query for a matching row using a parameterized rawQuery — the "?" placeholders keep user input out of the SQL string:

```
val db = DBHelper(this).readableDatabase
val cursor = db.rawQuery(
    "SELECT * FROM users WHERE email=? AND password=?",
    arrayOf(emailEditText.text.toString(), passwordEditText.text.toString())
)

if (cursor.moveToFirst()) {
    Toast.makeText(this, "Login success", Toast.LENGTH_SHORT).show()
} else {
    Toast.makeText(this, "Invalid credentials", Toast.LENGTH_SHORT).show()
}
cursor.close()
```

2. Add a third screen, UserListActivity, that reads every row and shows the usernames in a simple ListView or RecyclerView:

```
val cursor = db.rawQuery("SELECT * FROM users", null)
val names = mutableListOf<String>()
while (cursor.moveToNext()) {
    names.add(cursor.getString(cursor.getColumnIndexOrThrow("username")))
}
cursor.close()
```

3. Run the app on the emulator: register two or three users, then log in with correct and incorrect credentials, then open the user list and confirm all registered users appear.

> **Common pitfall**
> Forgetting to close the Cursor (and eventually the database) leaks resources. Call cursor.close() as soon as you're done reading, and prefer readableDatabase for pure lookups so you don't accidentally block writers.

Practice on your own

- Add a Delete Account button that removes the current user with db.delete("users", "email=?", arrayOf(email)).
- Add an Update Password screen using db.update(...) with a ContentValues object containing only the password column.
- Add a simple validation check (non-empty fields, basic email format) before calling insert(), and show an error rather than crashing.
- Bump the database version to 2 and add a phone column in onUpgrade() using ALTER TABLE — confirm existing rows survive the upgrade.

Self-check

- Can you explain, in one sentence, what onCreate() vs onUpgrade() is each responsible for?
- Why does the login query use “?” placeholders instead of string concatenation?

Activity 2 · Room: Student List

Storage layer	Room (abstraction over SQLite)
Builds on	Slides 12–18 — Entity, DAO, Database
You will practice	@Entity / @Dao / @Database, Room.databaseBuilder, suspend DAO functions, RecyclerView
Estimated time	60–75 minutes

Objective

Rebuild the same users table from Activity 1 using Room, then extend it into a small Student List app: add a student, list all students in a RecyclerView, and delete one. Notice how much boilerplate from Activity 1 disappears.

Step-by-step build

4. Create a new project “RoomStudentDemo” and add Room's dependencies to app/build.gradle.kts (Room compiler, Room KTX, and kapt/ksp plugin as recommended by the current Room release you install).
5. Create the Entity — a Kotlin data class annotated with @Entity, one property per column:

```
@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val username: String,
    val email: String,
    val password: String
)
```

6. Create the DAO — an interface listing the operations you need; Room generates the implementation and checks your SQL at compile time:

```
@Dao
interface UserDao {

    @Insert
    suspend fun insertUser(user: User)

    @Query("SELECT * FROM users")
    suspend fun getAllUsers(): List<User>

    @Query("""
        SELECT * FROM users
        WHERE email = :email AND password = :password
    """)
    suspend fun login(email: String, password: String): User?

    @Delete
    suspend fun deleteUser(user: User)
}
```

7. Create the Database class, listing the entities and exposing the DAO, then build a singleton instance (build it once, e.g. in an Application class or object, not on every screen):

```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}

val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java,
    "app_database"
).build()
```

8. Because DAO functions are suspend functions, call them from a coroutine scope such as lifecycleScope so they run off the main thread:

```
lifecycleScope.launch {
    db.userDao().insertUser(User(username = name, email = email, password = pass))
    val users = db.userDao().getAllUsers()
    // submit 'users' to your RecyclerView adapter
}
```

9. Build a RecyclerView with a simple adapter that shows username and email per row. After every insert or delete, re-query getAllUsers() and call adapter.submitList(...) / notifyDataSetChanged().
10. Wire up a delete icon or long-press on each row that calls db.userDao().deleteUser(user) for that row, then refreshes the list.
11. Run the app: add three or four students, confirm the list updates, delete one, and confirm it disappears.

Why Room over raw SQLite

Room validates your @Query strings against the Entity at compile time — a typo'd column name fails the build, not a user's login at runtime. It also removes manual Cursor-to-object mapping entirely.

Practice on your own

- Add an @Update DAO function and an “Edit student” flow that changes an existing row's email.
- Add a second Entity (e.g. Course) and a @Query that filters students by a search term using LIKE :query.
- Bump the database version and add a new column, then write a Migration object instead of letting Room destructively recreate the table.
- Replace the manual lifecycleScope calls with a ViewModel + Repository so the Activity holds no database logic at all.

Self-check

- Which of Entity, DAO, and Database is responsible for holding the DAO instances and configuring the database itself?
- What happens at compile time if you misspell a column name inside an @Query string?

Activity 3 · Cloud: Firestore Student Directory

Storage layer	Cloud Firestore (Firebase)
Builds on	Slides 19–23 — Firebase Android flow & Firestore reads
You will practice	Connecting a Firebase project, writing/reading documents, listeners, error states
Estimated time	60–90 minutes

Objective

Build the “Student Directory” described in the lecture's mini case study: a shared, cloud-backed list of students (name, registration number, department) that any authorized device can read and update, matching the lecture's Firebase Android Flow diagram (App → Firebase SDK → Cloud Database, gated by Security Rules).

Step-by-step build

12. In the Firebase console, create a new project, then register your Android app using its applicationId. Download google-services.json and place it in the app/ module folder.
13. Apply the Google Services Gradle plugin and add the Firestore dependency (via the Firebase BoM) to app/build.gradle.kts, then sync the project.
14. In the Firebase console, create a Cloud Firestore database in test mode for development, matching the lecture's users collection → document → fields model:

```
Collection: students
  Document: student_001
    name:            "Nimal"
    registrationNo:  "IT21001234"
    department:      "IT"
```

15. Get a Firestore instance and write a new student document from an “Add Student” form:

```
val db = FirebaseFirestore.getInstance()

val student = hashMapOf(
    "name" to nameEditText.text.toString(),
    "registrationNo" to regNoEditText.text.toString(),
    "department" to deptEditText.text.toString()
)

db.collection("students")
    .add(student)
    .addOnSuccessListener {
        Toast.makeText(this, "Student added", Toast.LENGTH_SHORT).show()
    }
    .addOnFailureListener { exception ->
        Toast.makeText(this, "Failed: ${exception.message}", Toast.LENGTH_SHORT).show()
    }
```

16. Read the whole collection into a RecyclerView, handling success and failure separately — never assume the network call succeeds:

```
db.collection("students")
    .get()
    .addOnSuccessListener { result ->
        val students = result.map { document ->
            Student(
                name = document.getString("name") ?: "",
                registrationNo = document.getString("registrationNo") ?: "",
                department = document.getString("department") ?: ""
            )
        }
        adapter.submitList(students)
    }
    .addOnFailureListener { exception ->
        // show a retry / error state instead of a blank screen
    }
```

17. (Optional, recommended) Replace the one-shot get() with an addSnapshotListener(...) so the RecyclerView updates automatically in real time when a document changes on another device.
18. In the Firebase console, tighten the Firestore Security Rules so only authenticated users can write, while reads remain open to the class — this maps directly to the lecture's “Security Rules” gate in the Firebase Android Flow diagram.
19. Run the app on two devices/emulators (or one emulator plus the Firebase console's data viewer): add a student on one, and confirm it appears when you refresh (or instantly, if you used a snapshot listener) on the other.

> **Common pitfall**
> Firestore calls are asynchronous and must never run on the assumption they'll finish before the next line executes. Always branch on addOnSuccessListener / addOnFailureListener, and move Firestore calls off the UI thread's critical path so a slow network doesn't freeze the screen.

Practice on your own

- Add a “Filter by department” dropdown backed by a Firestore .whereEqualTo("department", value) query.
- Add basic Firebase Authentication (email/password) and require sign-in before the Add Student screen is reachable.
- Show explicit loading, empty, offline, and error states in the UI rather than a blank RecyclerView while data is fetched.
- Enable Firestore's offline persistence and test the app in Airplane Mode — confirm previously loaded data is still visible.

Self-check

- In Firestore's data model, what is a Collection, what is a Document, and what are Fields?
- Why should the app never assume a Firestore write has already succeeded the line after you call it?

Capstone Practice · Offline-First Sync (Room + Firebase)

Storage layer	Room (local cache) + Firestore (cloud source), coordinated by a Repository
Builds on	Slides 24–30 — synchronization strategies & recommended architecture
You will practice	Repository pattern, source-of-truth decisions, conflict handling
Estimated time	Open-ended — self-directed extension

Objective

Combine Activities 2 and 3 into a single Student Directory that works offline: the UI always reads from Room, and a Repository quietly keeps Room and Firestore in sync, following the recommended architecture from the lecture (UI → ViewModel → Repository → Room / Firebase).

Suggested flow (mirrors the lecture's offline-first diagram)

20. User adds or edits a student — the Repository saves it to Room immediately so the UI updates without waiting on the network.
21. The Repository marks that row as “pending sync” (e.g. a boolean or timestamp column added to the Student entity).
22. When the device has network connectivity, the Repository pushes pending rows to Firestore.
23. The Repository also pulls remote changes from Firestore and updates Room to match.
24. If both Room and Firestore were changed for the same record, the Repository resolves the conflict using an explicit rule — do not leave this to chance.

> **Decide your conflict rule before you code**
> Pick one deliberately: last-write-wins by timestamp, cloud-always-wins, or field-level merging. The lecture's key point is that this must be an explicit business rule, not an accidental side effect of whichever write happens to finish last.

Practice on your own

- Add a syncStatus field to the Room Student entity (e.g. SYNCED, PENDING, CONFLICT) and surface it as a small icon in each RecyclerView row.
- Use a connectivity listener (or WorkManager) to trigger the push/pull sync only when the network becomes available, rather than polling constantly.
- Write a short design note (half a page) stating your chosen conflict-resolution rule and why it fits a student directory specifically.

Submission Checklist

Before submitting any activity, confirm the following:

- The app builds and runs on an emulator or device without crashing.
- CRUD operations were tested with at least three sample records, including an invalid case (e.g. wrong login, empty field).
- Cursor/database resources are closed where applicable (Activity 1).
- Compile-time query validation was demonstrated by intentionally breaking and fixing an @Query string once (Activity 2).
- Both success and failure paths are handled for every Firestore call (Activity 3).
- The “Practice on your own” tasks for each activity were attempted, even if incomplete — partial, documented attempts are expected.